

PROFESSIONAL SUMMARY

LLM serving control-plane engineer with five years of cloud-native AI/HPC experience and nearly three years focused on production model serving. Specializes in the layer between inference runtimes and production traffic: Kubernetes-native service lifecycle and Router/Worker readiness, OpenAI-compatible batch execution, gateway governance, observability, and failure recovery. Primary languages: Go and Python.

SELECTED IMPACT

~10M requests / day Production data-distillation and batch-generation path at >99% request success	300K requests / 9.75B tokens ~99.9% success in SGLang Router multi-backend testing	Ready → Routeable Desired-state reconciliation and readiness gates for Router worker convergence
--	--	--

EXPERIENCE

Zhejiang Lab · High-Performance Computing Facility Research Center2024 – Present

Senior Research Specialist | LLM Serving / MaaS Platform

- Developed a Kubernetes-native model-serving control path that maps model images, resource profiles, replica / prefill-decode (P/D) topology, and service exposure into CRDs and controllers; hardened Service/Endpoint readiness, finalizers, multi-cluster rollout, and rollback.
- Improved Router worker management by continuously reconciling desired workers against live /workers state and requeueing on missing, extra, or failed registrations, preventing “Pod Ready but traffic unreachable” failures.
- Designed and implemented an OpenAI Batch API-compatible execution platform with file and batch lifecycles, task sharding, sharded worker execution, Kubernetes Lease-based leader election, tenant concurrency, recovery, and result/error writeback.
- Supported a production data-distillation path at ~10M requests/day and >99% success; validated a SGLang Router multi-backend path at ~300K requests / 9.75B tokens and ~99.9% success.
- Built and operated unified model access with LiteLLM/Higress and SGLang/vLLM, covering authentication, rate limits, model access, streaming protocols, and incident diagnosis across request ID → Gateway → Router → Worker → Kubernetes.

Zhejiang Lab · Intelligent Supercomputing Research CenterAug 2021 – 2023

Engineering Specialist | Cloud-Native AI / HPC Platform

- Customized Kubernetes device-plugin and resource-management paths for heterogeneous accelerator discovery, node resource reporting, container-runtime validation, device-level allocation, and fault isolation.
- Adapted PyTorch/models and distributed job environments to Linux distributions and heterogeneous accelerator stacks, establishing the platform foundation for later GPU/XPU serving and scheduling work.

SELECTED SYSTEMS

Kubernetes-Native Model Serving Control Plane

Lifecycle · Reconciliation · Readiness

- Organized the model-service lifecycle around declarative desired state: controllers render and maintain workloads, Services, Router registration, and status, while ownership/finalizers, health checks, and retry budgets protect updates and cleanup.
- Separated instance startup from traffic readiness by jointly validating Pods, Endpoints, model APIs, and Router worker state; established recovery paths for failed registration, stale selectors, and long-lived connection failures.
- Worked on per-service scaling guardrails around resource snapshots, readiness, in-flight work, and cooldown; further separated Planner capacity intent, replica execution, cluster constraints, and Router readiness in a proposed control-plane evolution.

Multi-Tenant Batch Inference Platform

OpenAI Batch API · Scheduling · Recovery

- Modeled batches as durable, recoverable asynchronous jobs rather than bulk request scripts: the database owns task truth while schedulers/workers claim, shard, execute, renew, and finalize work.
- Enabled multi-replica operation with Kubernetes Lease leadership, database-level claims, tenant/model concurrency budgets, and request-level error isolation, preventing duplicate execution and limiting offline impact on online serving.
- Implemented streaming JSONL processing, cancellation, expiry cleanup, output/error files, and failure-tolerant finalization from API contract through production monitoring.

Model Gateway, Routing, and Serving Reliability

Gateway · Router · Observability

- Provided OpenAI-compatible access through LiteLLM/Higress with API keys, team/model access, TPM/RPM limits, token budgets, model mapping, and request auditability.
- Benchmarked SGLang/vLLM P/D topology and cache-aware routing with QPS, TTFT, and TPOT; traced cache-hit degradation to dynamic fields breaking shared prompt prefixes.
- Handled production troubleshooting and release verification for Router registration, Service/Endpoint readiness, CrashLoops, streaming connections, multi-cluster configuration, and rollback.

CORE TECHNOLOGIES

Serving Control Plane	Go, Kubernetes, CRDs/controllers, reconciliation, Service/EndpointSlice, readiness, finalizers, leader election, Helm, Kustomize, ArgoCD
Inference & Routing	SGLang, vLLM, Router, P/D disaggregation, KV/prefix cache, inference performance metrics
Platform & Data Path	Python, Linux, OpenAI-compatible APIs, LiteLLM, Higress, Prometheus, Redis, MySQL, request tracing, distributed-systems debugging
Upstream Architecture	NVIDIA Dynamo, AIBrix, llm-d: capacity intent, replica execution, routing readiness, and cluster-constraint boundaries (source-level evaluation)

EDUCATION

Arizona State University	May 2019 - Dec 2020
Master of Computer Science, GPA 3.97 / 4.00	
Arizona State University	Aug 2015 - May 2019
B.S. in Computer Science	

INTELLECTUAL PROPERTY

Co-inventor on two granted invention patents, including an elastic cloud model-inference serving system; contributor to two authorized software copyrights.